1. System Abstract

Modern agriculture faces severe vulnerabilities due to climate instability, soil nutrient degradation, and unpredictable rainfall patterns. Traditionally, farmers select crops based on historical intuition rather than real-time chemical parameters, leading to depleted soil health and reduced crop yields.

Opti-Crop is an advanced data-driven agricultural platform designed to resolve crop selection uncertainties and mitigate the risk of crop failure. The system integrates soil parameters with local meteorological indicators to deliver precise, scientific recommendations. Specifically, the model processes seven indicators:

- **Nitrogen (N):** Essential for leaf growth and vegetative development.
- **Phosphorous (P):** Critical for root development and early plant maturation.
- **Potassium (K):** Increases disease resistance and overall crop quality.
- **Soil pH:** Regulates nutrient availability and microbial soil activity.
- **Temperature (°C):** Dictates metabolic rates and growth suitability thresholds.
- **Relative Humidity (%):** Influences transpiration rates and moisture retention.
- **Rainfall (mm):** Represents water requirements and irrigation dependencies.

Aesthetic System Objective: *Beyond crop recommendation, Opti-Crop generates tailored organic and chemical soil correction guidelines to assist users in rebalancing soil nutrient ratios.*

2. Solution Architecture

Opti-Crop implements a modular Model-View-Controller (MVC) architecture, separating client interaction, web-routing logic, and the scientific machine learning core. The application processes soil queries synchronously and dynamically renders customized agronomical dashboards.

Key Logical Components:

1. **Flask Web Server (app.py):** Exposes RESTful endpoints, handles security session boundaries (cookies, password hashing), and manages form submissions.
2. **Database Access (database.py):** Construct schemas, configures PRAGMA configurations (foreign keys), and injects default crop datasets.
3. **Machine Learning Engine (train_model.py):** Implements statistical outlier treatment, trains the predictive engine, performs clustering, and serializes state pickles.
4. **Presentation Layer (templates/ & static/):** Utilizes custom typography, responsive design wrappers, and CSS grid frameworks.

3. Relational Database Schema

The system utilizes SQLite3 for state persistence. The database consists of 7 tightly coupled tables engineered with Cascade Deletion policies to maintain database integrity across actions.

Table Name	Primary Key	Columns	Foreign Keys / Relations
users	user_id	username (TEXT, Unique) email (TEXT, Unique) password (TEXT, Hashed) role (TEXT)	Parent reference for soil submissions.
soil_data	soil_id	nitrogen (REAL) phosphorous (REAL) potassium (REAL) temperature (REAL) humidity (REAL) ph (REAL) rainfall (REAL) season (TEXT) user_id (INTEGER)	user_id references users(user_id) on delete set null
crops	crop_id	crop_name (TEXT, Unique) crop_type (TEXT) season (TEXT) optimal_ph (REAL) water_requirement (REAL)	Standard parameters table for 22 crops.
datasets	dataset_id	dataset_name (TEXT) source (TEXT) total_records (INTEGER) last_updated (TEXT)	Reference metadata dataset details.
ml_models	model_id	model_name (TEXT) accuracy (REAL) dataset_id (INTEGER)	dataset_id references datasets(dataset_id) on delete cascade
predictions	prediction_id	soil_id (INTEGER) crop_id (INTEGER) model_id (INTEGER) prediction_date (TIMESTAMP) confidence_score (REAL)	soil_id references soil_data crop_id references crops model_id references ml_models on delete cascade
reports	report_id	prediction_id (INTEGER) summary (TEXT) recommendations (TEXT)	prediction_id references predictions on delete cascade

4. Route & Endpoint Configurations

The system exposes routes for page rendering (GET) and data processing pipelines (POST):

Endpoint	Method	Description	Inputs / Outputs
/	GET	Renders home dashboard page.	None
/findyourcrop	GET	Renders soil metric input forms.	None
/predict	POST	Performs crop predictions and saves soil metadata.	Form: N, P, K, pH, temp, humidity, rainfall Output: Predicted Crop & Soil Advice
/suitability	GET	Renders target crop compatibility analyzer UI.	None
/evaluate_suitability	POST	Evaluates soil inputs against target crop std deviations.	Form: Crop Name, Soil Parameters Output: Compatibility Score & Advice
/insights	GET	Visualizes KMeans clustering groupings & model performance.	Plots loaded dynamically from static directory.
/register	GET/POST	User onboarding and credentials hashing.	Form: Username, Email, Password, Role
/login	GET/POST	Establishes secure authentication cookie sessions.	Form: Username, Password

5. Machine Learning Methodology

The application leverages a two-phase machine learning pipeline combining supervised classification and unsupervised clustering to provide robust agronomic insights.

Outlier Mitigation & IQR

To ensure high generalization, outliers are removed from the Phosphorous column in the training dataset. The Interquartile Range (IQR) method calculates Q1 (25th percentile) and Q3 (75th percentile). Data points falling outside 1.5 times the IQR range are scrubbed, ensuring the model's coefficients are not skewed.

Supervised Classification: Logistic Regression

Using the preprocessed 7 features, a multi-class Logistic Regression classifier is trained. Using an 80/20 train-test split, the classifier achieves a cross-validated prediction accuracy of **94.0%** across the 22 crops. The trained model state is exported to *model.pkl*.

Unsupervised Clustering: K-Means Clustering

To discover soil parameter boundaries without labeled target outputs, the system fits a K-Means model (K=4). This identifies general chemical environments (e.g. high-nitrogen/low-rainfall vs. low-nitrogen/high-rainfall). The Elbow Method (WCSS plot) is executed and saved to *static/images/elbow_graph.png* on model retraining, allowing developers to audit model groupings over time.

6. Local Installation & Administration Guide

Execute these terminal commands in order to set up and run the system locally:

```
# Step 1: Clone Repository
git clone https://github.com/AmbicaSairam/Opti-Crop.git
cd Opti-Crop
```

```
# Step 2: Configure Virtual Environment
python -m venv venv
.\venv\Scripts\activate  # On Windows
source venv/bin/activate    # On Linux/macOS
```

```
# Step 3: Install Required Dependencies
pip install -r "5. Project Development Phase/requirements.txt"
```

```
# Step 4: Setup Database & Generate Seed Data
python "5. Project Development Phase/database.py"
```

```
# Step 5: Retrain Model & Generate Graphs
python "5. Project Development Phase/train_model.py"
```

```
# Step 6: Start Flask Server
python "5. Project Development Phase/app.py"
```

7. Cloud Deployment Configurations

Opti-Crop includes a *render.yaml* blueprint to enable automatic, multi-stage cloud builds. The blueprint compiles database seed files and executes the training pipeline before starting the web listener.

```
services:
  - type: web
    name: opticrop
    env: python
    buildCommand: pip install -r "5. Project Development Phase/requirements.txt" && python "5. P
roject Development Phase/database.py" && python "5. Project Development Phase/train_model.py"
    startCommand: gunicorn "5. Project Development Phase.app:app"
```